Digital Chip Design Program

RTL Design

Almas Alshubrumi
Atheer Al-harbi
Mjd AL-harbi
Maha Omar
Jood Alomair

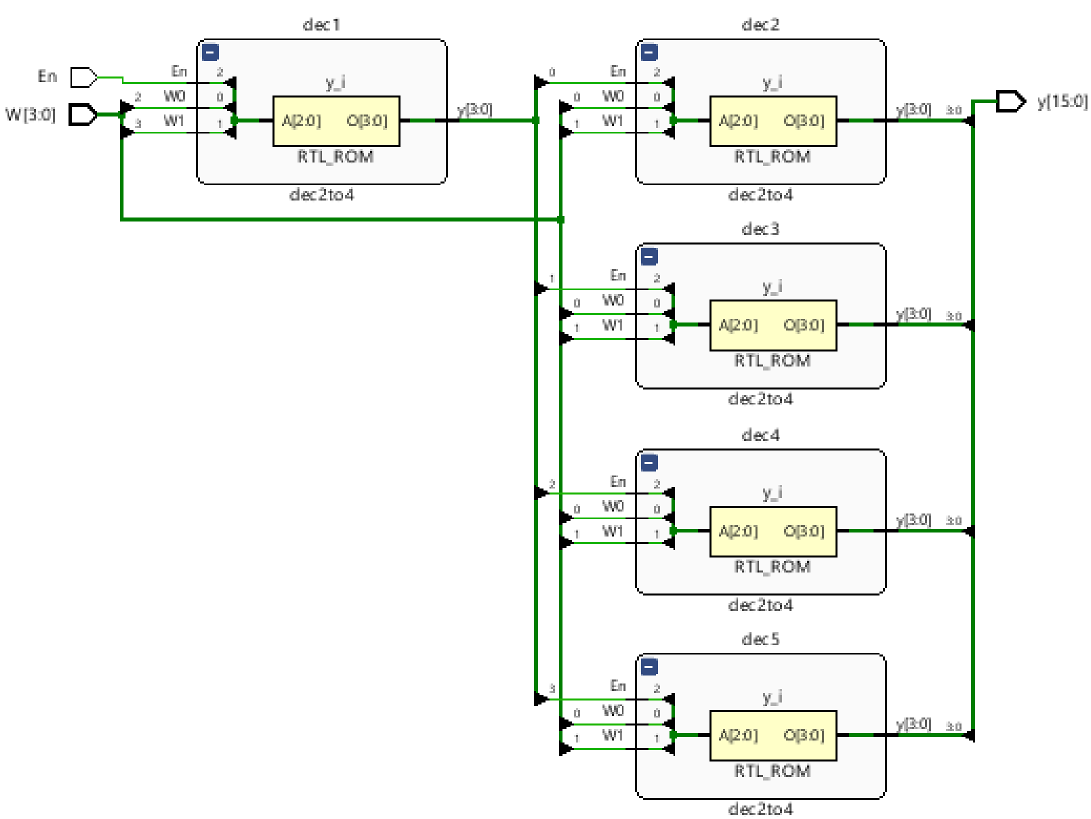
7/7/2026

Combinational Datapath Design Part 1

Task #5.1

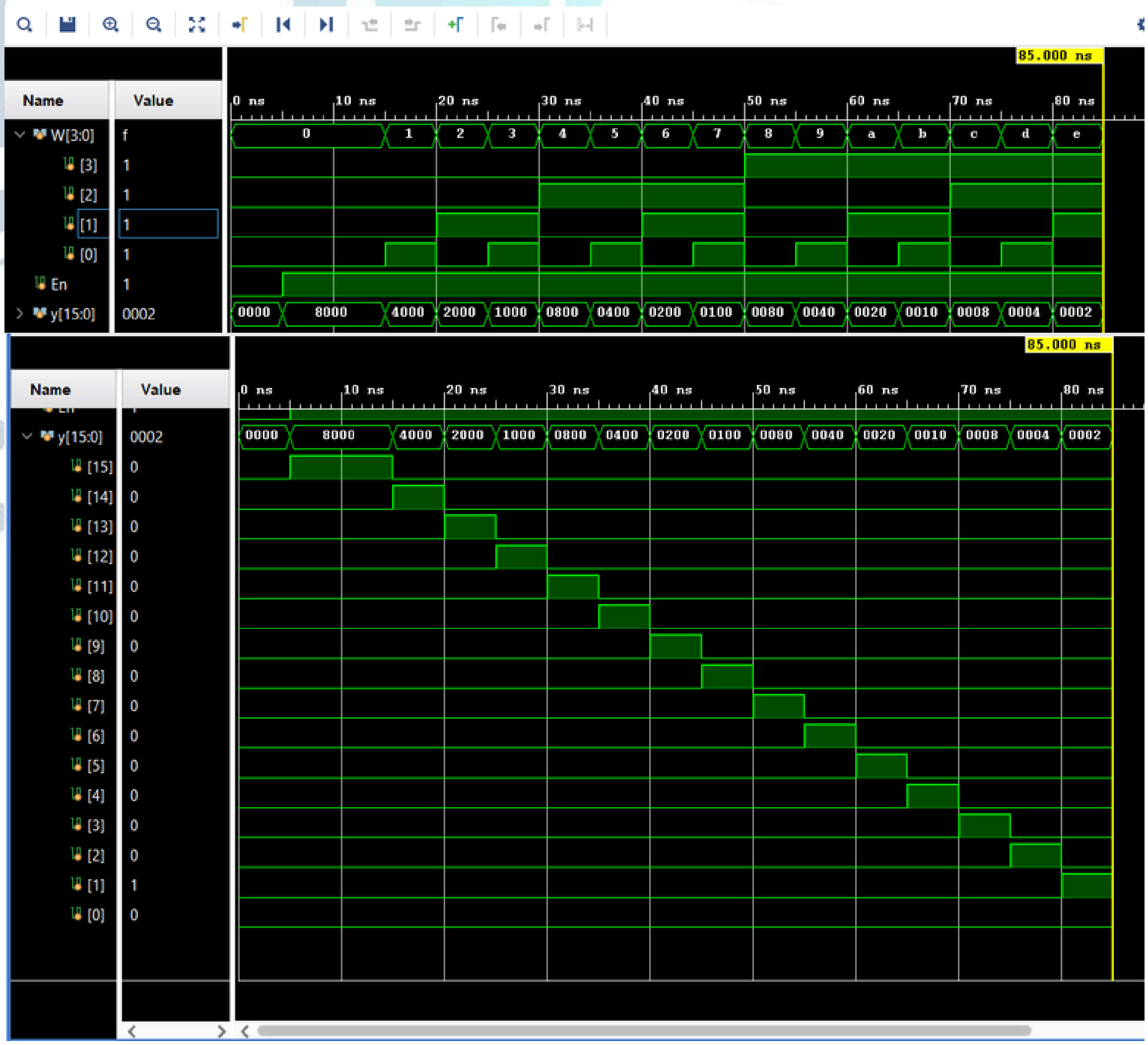
```
module dec2to4(
input logic W0,W1,En,
output logic [3:0] y
 );
 always_comb
case({En,W1,W0})
3'b100:y=4'b1000;
3'b101:y=4'b0100;
3'b110:y=4'b0010;
3'b111:y=4'b0001;
default:y=4'b0000;

endcase
endmodule
```

```
module dec4to16(
input logic [3:0] W,
input logic En,
output logic [15:0] y
 );
 logic [3:0] m;
 dec2to4 dec1(.W0(W[2]),.W1(W[3]),.En(En),.y(m[3:0])) ;
 dec2to4 dec2(.W0(W[0]),.W1(W[1]),.En(m[0]),.y(y[3:0])) ;
 dec2to4 dec3(.W0(W[0]),.W1(W[1]),.En(m[1]),.y(y[7:4])) ;
 dec2to4 dec4(.W0(W[0]),.W1(W[1]),.En(m[2]),.y(y[11:8])) ;
 dec2to4 dec5(.W0(W[0]),.W1(W[1]),.En(m[3]),.y(y[15:12]))
;
endmodule
```



Task #5.1

```
`timescale 1ns / 1ps
module dec4to16_tb();
logic [3:0] W;
logic En ;
logic [15:0]y;
dec4to16 dut(
.W(W),
.En(En),
.y(y));
initial begin
En=0 ;
W=4'b0000; #5
W=0;
En=1;
for(int i=0;i<16;i++)
#5 W=i;
$finish ;
end
endmodule
```



Task #5.2

2.1

```
module half_adder(
 input A,
 input B,
 output S,
 output Cout);
assign S = A ^ B;
assign Cout = A & B;
endmodule
```

2.2

```
module full_adder(
 input A,B,Cin,
 output S, Cout);
wire S1,C1,C2;
half_adder HA1(
.A(A),
.B(B),
.S(S1),
.Cout(C1));

half_adder HA2(
.A(S1),
.B(Cin),
.S(S),
.Cout(C2));
assign Cout = C1 | C2;
endmodule
```

2.3

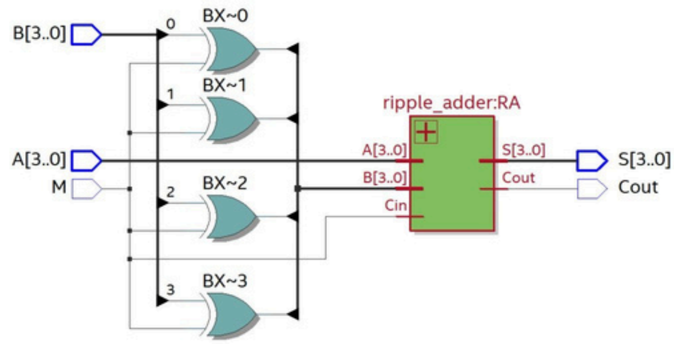
```
module ripple_adder(
 input logic [3:0] A,
 input logic [3:0] B,
 input logic Cin,
 output logic [3:0] S,
 output logic Cout);
logic c1; logic c2; logic c3;
full_adder FA0(
 .A(A[0]), .B(B[0]), .Cin(Cin), .S(S[0]), .Cout(c1));
full_adder FA1( .A(A[1]), .B(B[1]), .Cin(c1), .S(S[1]),
.Cout(c2));
full_adder FA2( .A(A[2]), .B(B[2]), .Cin(c2), .S(S[2]),
 .Cout(c3));
full_adder FA3(
 .A(A[3]), .B(B[3]),
 .Cin(c3), .S(S[3]), .Cout(Cout));
endmodule
```

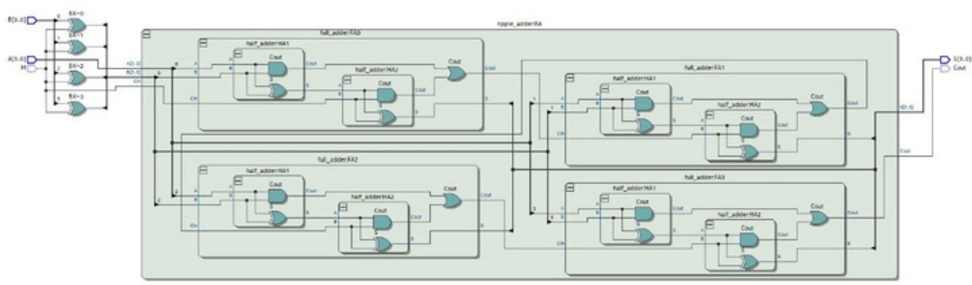
2.4

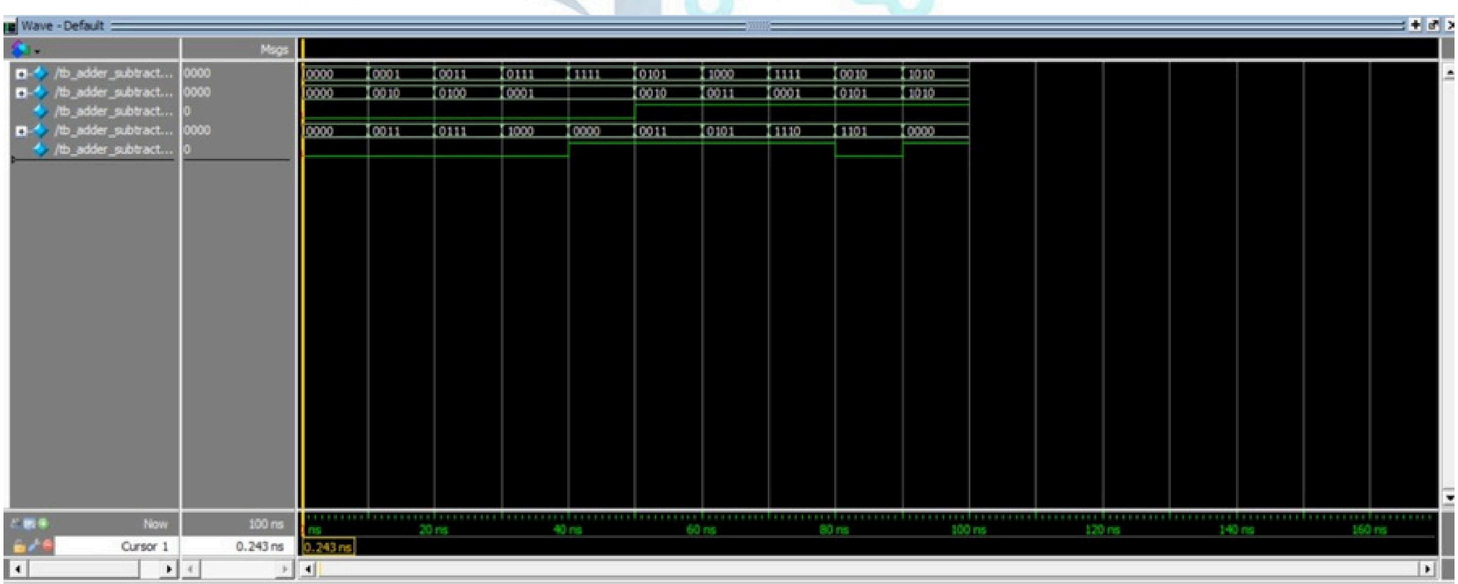
```
module adder_subtractor(
 input logic [3:0] A,
 input logic [3:0] B,
 input logic M,
 output logic [3:0] S,
 output logic Cout );
logic [3:0] BX;
assign BX = B ^ {4{M}};
ripple_adder RA(
 .A(A), .B(BX), .Cin(M), .S(S),
 .Cout(Cout));
endmodule
```

Task #5.2

```
`timescale 1ns/1ps
module tb_adder_subtractor;
logic [3:0] A;
logic [3:0] B;
logic M;
logic [3:0] S;
logic Cout;
adder_subtractor DUT(
.A(A), .B(B), .M(M), .S(S), .Cout(Cout));
initial begin
M = 0;
A = 4'b0000; B = 4'b0000; #10;
A = 4'b0001; B = 4'b0010; #10;
A = 4'b0011; B = 4'b0100; #10;
A = 4'b0111; B = 4'b0001; #10;
A = 4'b1111; B = 4'b0001; #10;
// Subtraction
M = 1;
A = 4'b0101; B = 4'b0010; #10;
A = 4'b1000; B = 4'b0011; #10;
A = 4'b1111; B = 4'b0001; #10;
A = 4'b0010; B = 4'b0101; #10;
A = 4'b1010; B = 4'b1010; #10;
$finish;
end
endmodule
```

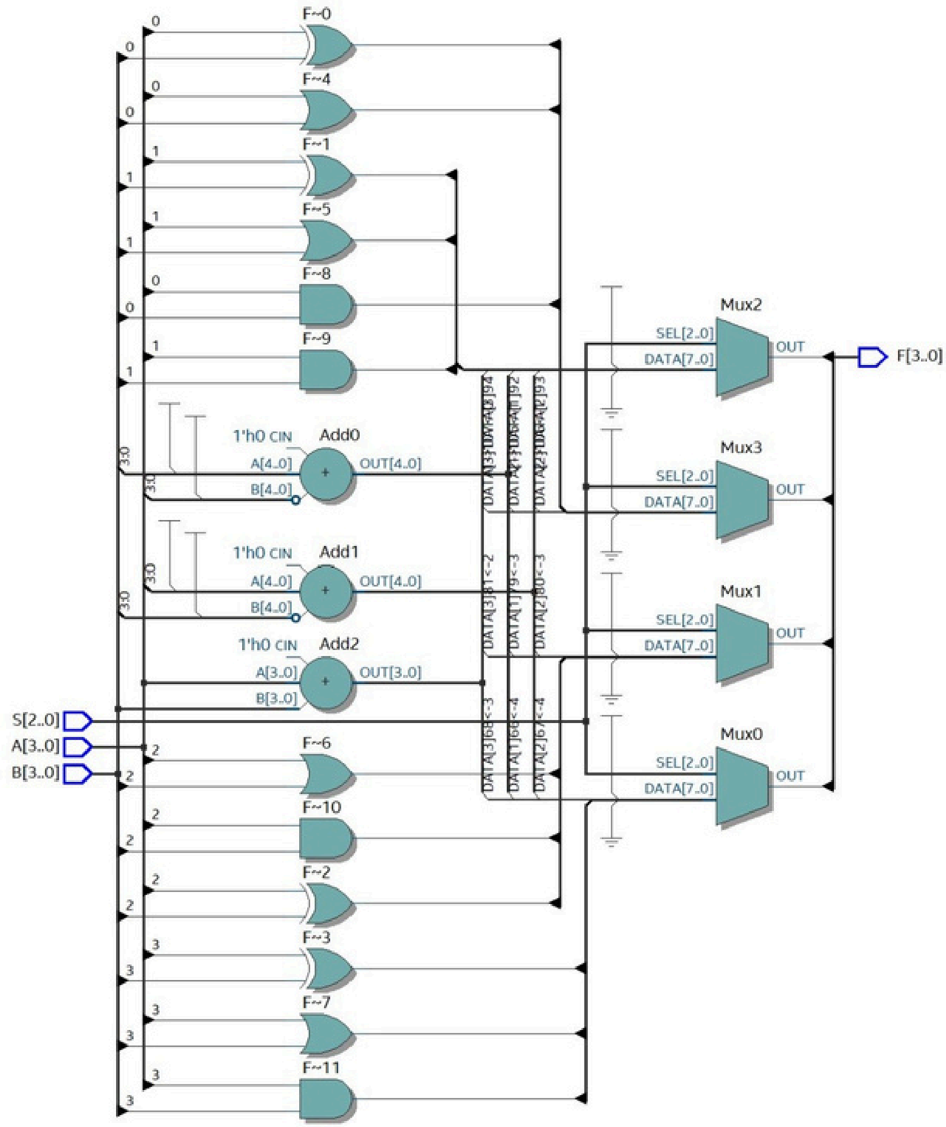






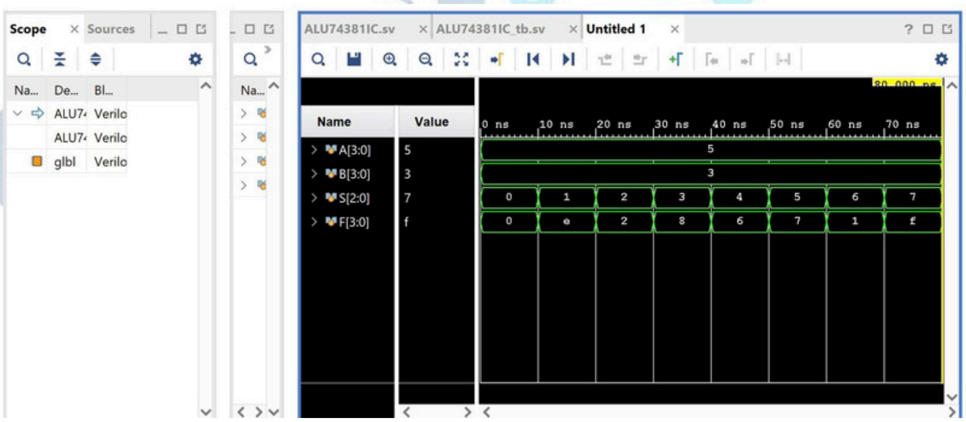
Task #5.3

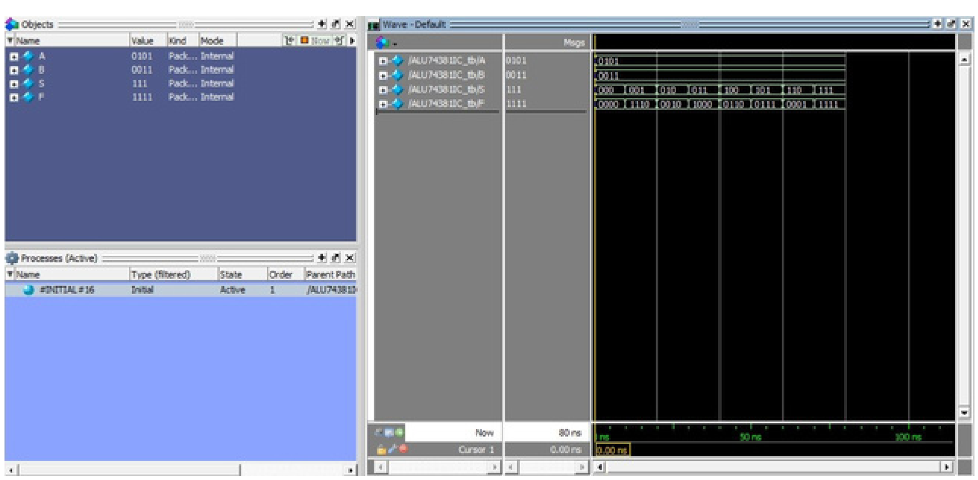
```
module ALU74381IC(
  input logic [3:0] A,
  input logic [3:0] B,
  input logic [2:0] S,
  output logic [3:0] F
);
always_comb begin
  case (S)
    3'b000: F = 4'b0000;

    3'b001: F = B - A;

    3'b010: F = A - B;  // A - B

    3'b011: F = A + B;  // Addition

    3'b100: F = A ^ B;  // XOR

    3'b101: F = A | B;  // OR

    3'b110: F = A & B;  // AND

    3'b111: F = 4'b1111; // Preset

    default: F = 4'b0000;
  endcase
end

endmodule
```



Task #5.3

```
`timescale 1ns/1ps
module ALU74381IC_tb;
logic [3:0] A, B;
logic [2:0] S;
logic [3:0] F;
ALU74381IC dut (
  .A(A),
  .B(B),
  .S(S),
  .F(F)
);
initial begin
  A = 4'b0101;  // 5
  B = 4'b0011;  // 3
  S = 3'b000;
  #10;
  S = 3'b001;
  #10;
  S = 3'b010;
  #10;
  S = 3'b011;
  #10;
  S = 3'b100;
  #10;
  S = 3'b101;
  #10;
  S = 3'b110;
  #10;
  S = 3'b111;
  #10;

  $finish;

end

endmodule
```





Work distribution

Section	Almas	Atheer	Mjd	Maha	Jood
Report	✔	✔	✔	✔	✔
Task5.1	✔	✔	✔	✔	✔
Task5.2	✔	✔	✔	✔	✔
Task5.3	✔	✔	✔	✔	✔